

Pyth Network on Algorand — A Senior Architect's Map from Hermes Off-Chain Reads to Confidence-Hardened Pull Updates

TL;DR

- **There is no native Pyth Network contract on Algorand.** Pyth's canonical contract-address registry at <https://docs.pyth.network/price-feeds/core/contract-addresses> enumerates EVM, Solana/SVM, Aptos, Sui, IOTA, Movement, TON, Fuel, CosmWasm, NEAR, Starknet and Pythnet — Algorand is absent — and the [pyth-network/pyth-crosschain](https://github.com/pyth-network/pyth-crosschain) monorepo contains no [target_chains/algorand/](https://github.com/pyth-network/pyth-crosschain/tree/main/target_chains) directory (confirm by browsing https://github.com/pyth-network/pyth-crosschain/tree/main/target_chains). Pyth Lazer/Pyth Pro and Pyth Entropy likewise have no Algorand build. The Pyth Network blog post "Pyth's Next Steps | 2022" (<https://www.pyth.network/blog/pyths-next-steps-2022>, October 2022) stated verbatim "Up next are all of your favorite L1s and L2s, including Ethereum, Polygon, Injective, NEAR, Avalanche, Fantom, Algorand, Arbitrum, Optimism, Aptos, Sui and more. 'If it's fair game for Wormhole, it's fair game for Pyth.'" — but that Algorand build never shipped. [Pyth](#) [Pyth Network](#)
- **Therefore the only viable Pyth-on-Algorand integrations today are off-chain Hermes reads (lowest cost, zero on-chain integrity) and a custom, do-it-yourself pull-receiver that consumes Pyth's Wormhole-signed price VAAs through the existing Wormhole Core app on Algorand (mainnet App ID [842125965](#)).** This is exactly the architecture C3.io uses — a private, application-specific Pyth verifier sitting on top of [wormhole-foundation/wormhole/algorand/wormhole_core.py](#) and [vaa_verify.py](#). [Go Packages](#)
- **Concrete recommendation for PYTHAI / DELTAVERSE / BANKON:** for non-funds-gating UI and mindX-agent reasoning use Hermes REST/SSE directly from Python (Tier 0, free, sub-second); for funds-gating logic on Algorand build a DELTAVERSE-specific Pyth receiver in Algorand Python (Algopy) that calls into Wormhole Core App [842125965](#) to verify VAAs and stores [\(price, conf, expo, publish_time\)](#) in box storage with hard staleness and confidence-ratio rejects, and cross-check against a second oracle (Folks Finance feed or a Tinyman TWAP) before settling DebasementIndexV2 trades. Do not assume Pyth Algorand support will materialize — write to the Wormhole VAA wire format directly.

Key Findings

1. The canonical surface of [pyth-network/pyth-crosschain](https://github.com/pyth-network/pyth-crosschain)

The monorepo lives at <https://github.com/pyth-network/pyth-crosschain>. Its [target_chains/](https://github.com/pyth-network/pyth-crosschain/tree/main/target_chains/) directory (browsable at https://github.com/pyth-network/pyth-crosschain/tree/main/target_chains/)

`crosschain/tree/main/target_chains`) contains one subdirectory per supported runtime: `ethereum/`, `solana/`, `aptos/`, `sui/`, `cosmwasm/`, `near/`, `ton/`, `fuel/`, `starknet/`, `iota/`, `movement/`. **There is no `algorand/` subdirectory in `target_chains/`, in `lazer/contracts/`, or in `contract_manager/`.** The same conclusion is reached via the docs site: `https://docs.pyth.network/price-feeds/core/contract-addresses` only links to per-ecosystem pages for EVM, Solana, Aptos, Sui, IOTA, Movement, TON, Fuel, CosmWasm, NEAR, Starknet, Pythnet. `(pyth)` `(pyth)`

The repository's top-level layout (`apps`, `contract_manager`, `governance`, `lazer/contracts`, `packages`, `price_service`, `pythnet`, `target_chains`) is confirmed by the README at `https://github.com/pyth-network/pyth-crosschain/blob/main/README.md`. The Hermes service source code is at `https://github.com/pyth-network/pyth-crosschain/tree/main/apps/hermes`. The Lazer (Pyth Pro) contracts are at `https://github.com/pyth-network/pyth-crosschain/tree/main/lazer/contracts` and only contain `evm/`, `solana/`, `sui/`, `fogo/` subtrees — **no Algorand Lazer contract**. Pyth Entropy is documented as EVM-only at `https://docs.pyth.network/entropy/contract-addresses`, with the canonical Entropy mainnet provider at the address `0x52DeaA1c84233F7bb8C8A45baeDE41091c616506` and contracts in `https://github.com/pyth-crosschain/tree/main/target\_chains/ethereum/contracts/contracts/entropy`. `(Pyth)`

2. The Wormhole substrate that any Algorand-side Pyth integration must use

Because Pyth's cross-chain data is delivered as Wormhole VAAs from Pythnet (see `https://docs.pyth.network/price-feeds/core/how-pyth-works/cross-chain`), any on-chain Algorand verification must go through Wormhole's existing Algorand Core. The PyTeal source lives in `wormhole-foundation/wormhole/algorand/`, browsable at `https://github.com/wormhole-foundation/wormhole/tree/main/algorand`:

- Core stateful app: `https://github.com/wormhole-foundation/wormhole/blob/main/algorand/wormhole\_core.py` `(GitHub)`
- Stateless guardian-signature verifier (`VaaVerify`): `https://github.com/wormhole-foundation/wormhole/blob/main/algorand/vaa\_verify.py` `(GitHub)`
- Dynamic storage stateless template (`TmplSig`): `https://github.com/wormhole-foundation/wormhole/blob/main/algorand/TmplSig.py` `(GitHub)`
- Token Bridge: `https://github.com/wormhole-foundation/wormhole/blob/main/algorand/token\_bridge.py` `(GitHub)`

The deployed App IDs are documented at

`https://wormhole.com/docs/products/reference/contract-addresses/`:

- **Algorand mainnet — Wormhole Core App ID:** `842125965`

- **Algorand mainnet — Token Bridge / Portal App ID:** `842126029`
- **Algorand testnet — Wormhole Core App ID:** `86525623`
- **Algorand testnet — Token Bridge App ID:** `86525641` `wormhole`
- Algorand's Wormhole chain ID is `8`.

The dominant attack surface here is that Pyth-emitted VAAs from Pythnet carry a specific emitter chain ID and emitter address (the Wormhole emitter on Pythnet, which is itself the Pyth oracle program). Any custom Algorand receiver must (a) call Wormhole Core's VAA-verification path to confirm guardian signatures, (b) gate on the expected Pythnet emitter (chain 26, the Pyth-Wormhole emitter address), and (c) parse Pyth's Merkle-rooted price message payload itself — none of which Wormhole Core does for you, since `wormhole_core.py` is payload-agnostic.

3. The C3.io reference: how Pyth-on-Algorand actually works in production

C3.io is the only sizeable Algorand protocol that publicly claims to use Pyth, and the Pyth blog post at <https://pyth.network/blog/the-best-of-defi-and-cefi-with-c3-interview> plus the C3 technical overview at <https://blog.c3.io/technical-overview-of-c3-everything-you-need-to-know/> and the Pyth case study at <https://www.pyth.network/blog/a-new-milestone-in-price-updates-with-c3-ios-successful-testnet-pyth-case-study> together make the architecture clear: C3's on-chain component on Algorand consists of a Cross-collateral Clearing Engine and a Health Calculator stateful contract, and the Health Calculator "taps into Pyth Price Feeds to establish the value of assets per account at ultra-low latencies." C3 is also a Wormhole consumer (its Portal Bridge integration is documented at <https://wormhole.com/c3-case-study/>), and since Pyth price updates on every non-Solana chain are Wormhole VAAs, the most parsimonious — and only architecturally consistent — explanation is that C3 verifies Pyth VAAs on Algorand using the same Wormhole Core App `842125965` it already integrates for token transfers. There is no published C3-Pyth-Algorand receiver contract repository, so this remains the canonical pattern but a closed-source one. Treat it as a proof of feasibility, not as a drop-in dependency. `Pyth Network + 2`

4. Pyth Lazer and Pyth Entropy: not on Algorand

Pyth Lazer (now branded Pyth Pro for the highest-frequency tier) is documented at <https://docs.pyth.network/price-feeds/pro> and the contract source tree at <https://github.com/pyth-network/pyth-crosschain/tree/main/lazer/contracts> shows that supported runtimes are Solana, EVM, Sui, and Fogo. Algorand is not listed as a Lazer target. Pyth Entropy V2 is EVM-only per <https://docs.pyth.network/entropy/contract-addresses>; the `IEntropy` Solidity interface and the `entropyCallback` pattern have no Algorand analogue. **For DAIO randomness on Algorand the canonical answer is not Pyth Entropy but Algorand's own VRF (`vrf_verify` opcode) combined with the Algorand Foundation's Randomness**

Beacon application, or an off-chain Pyth Entropy consumer reached via cross-chain messaging.

5. PYTH token on Algorand

The PYTH token is a native Solana SPL token. Pyth's own token-addresses page at <https://docs.pyth.network/home/pyth-token/pyth-token-addresses> lists the Solana mint plus EVM deployments; no Algorand ASA is enumerated, and the Wormhole token list at <https://github.com/wormhole-foundation/wormhole-token-list> does not contain a canonical PYTH ASA either. While the Wormhole Token Bridge on Algorand (App [842126029](#)) is technically permissionless and anyone may attest a PYTH wrapper into Algorand as an ASA, **there is no officially recognized PYTH ASA**. Oracle Integrity Staking (OIS), the slashing-backed staker-behind-publisher system, is exclusively a Solana / Pythnet program — the user interface is at <https://staking.pyth.network/> and the on-chain program source is in <https://github.com/pyth-network/governance> (specifically [staking/programs/staking](#)). Any DELTAVERSE staking exposure to OIS must therefore bridge PYTH from Solana through Portal Bridge (<https://portalbridge.com>) directly on the Solana side, not from Algorand.

[Pyth](#)

Details: A Cost-Ordered Map of Pyth-on-Algorand Surface Area

Tier 0 — Hermes off-chain reads (zero on-chain cost, zero on-chain integrity)

The lowest-cost and lowest-integrity tier is to ignore Algorand entirely and consume Pyth prices off-chain via Hermes, the public price service operated by the Pyth Data Association. The endpoints are documented at <https://docs.pyth.network/price-feeds/core/fetch-price-updates> and the source code is at <https://github.com/pyth-network/pyth-crosschain/tree/main/apps/hermes>. The production endpoint is <https://hermes.pyth.network>; a beta/Sepolia equivalent is <https://hermes-beta.pyth.network>. Pyth's published refresh cadence is up to 400 ms on chains that support sub-second updates (Solana); Hermes itself exposes the most recent observed update with no intentional delay, but a defensible upper-bound on freshness is one Pythnet slot plus a Wormhole guardian round, on the order of hundreds of milliseconds. [CoinGecko](#) [OAK Research](#)

The REST surface that matters for a Python 3.12 + Algopy stack:

- [GET https://hermes.pyth.network/v2/updates/price/latest?ids\[\]=<feed_id_hex>](https://hermes.pyth.network/v2/updates/price/latest?ids[]=<feed_id_hex>) returns the latest price, confidence, exponent, publish time, and a base64-encoded VAA blob ready for on-chain submission. [Pyth](#)
- [GET https://hermes.pyth.network/v2/updates/price/stream?ids\[\]=<feed_id_hex>](https://hermes.pyth.network/v2/updates/price/stream?ids[]=<feed_id_hex>) is an SSE stream for continuous updates, suitable for mindX agent live ticker feeds.
- [GET https://hermes.pyth.network/v2/price_feeds?asset_type=crypto&query=btc](https://hermes.pyth.network/v2/price_feeds?asset_type=crypto&query=btc) enumerates feed IDs by query — this is the programmatic equivalent of

<https://www.pyth.network/developers/price-feed-ids>.

Reference price feed IDs (Stable channel, mainnet):

- BTC/USD: [0xe62df6c8b4a85fe1a67db44dc12de5db330f7ac66b72dc658afedf0f4a415b43](#)
[Pyth](#) [Medium](#)
- ETH/USD: [0xff61491a931112ddf1bd8147cd1b641375f79f5825126d665480874634fd0ace](#)
[Pyth](#) [Medium](#)
- SOL/USD: [0xef0d8b6fda2ceba41da15d4095d1da392a0d2f8ed0c6c7bc0f4cfac8c280b56d](#)
[Medium](#)
- AVAX/USD: [0x93da3352f9f1d105fdfe4971cfa80e9dd777bfc5d0f683ebb6e1294b92137bb7](#)
[Medium](#)
- ALGO/USD, USDC/USD, EUR/USD, USD/JPY, GBP/USD, XAU/USD all exist as feeds (see the directory at <https://www.pyth.network/price-feeds>) but their exact 32-byte IDs should be retrieved at integration time from https://hermes.pyth.network/v2/price_feeds to avoid stale-ID drift across Stable vs. Beta channels.

The official JavaScript Hermes client is published as [@pythnetwork/hermes-client](#) (source in <https://github.com/pyth-network/pyth-crosschain/tree/main/apps/hermes>). For Python ≥ 3.12 there is no Pyth-published Hermes-V2 client; the practical answer is to use [httpx](#) or [aiohttp](#) directly. The legacy [pyth-client-py](#) repo at <https://github.com/pyth-network/pyth-client-py> reads on-chain Solana Pyth accounts and is not what you want for Hermes.

A minimal [pythai/oracle/hermes.py](#) skeleton:

```
python
```

```

# pythai/oracle/hermes.py
import asyncio, base64, time
import httpx

HERMES = "https://hermes.pyth.network"
BTC_USD = "0xe62df6c8b4a85fe1a67db44dc12de5db330f7ac66b72dc658afedf0f4a415b43"

async def get_price(feed_id: str) -> dict:
    async with httpx.AsyncClient(timeout=2.0) as c:
        r = await c.get(
            f"{HERMES}/v2/updates/price/latest",
            params={"ids[]": feed_id, "encoding": "base64"},
        )
        r.raise_for_status()
        body = r.json()
        # body["parsed"][0] = {"id", "price": {"price", "conf", "expo", "publish_time"}}
        # body["binary"]["data"][0] = base64 VAA blob (use this for on-chain submit)
        parsed = body["parsed"][0]["price"]
        return {
            "feed_id": feed_id,
            "price": int(parsed["price"]),
            "conf": int(parsed["conf"]),
            "expo": int(parsed["expo"]),
            "publish_time": int(parsed["publish_time"]),
            "vaa_b64": body["binary"]["data"][0],
            "age_s": time.time() - int(parsed["publish_time"]),
        }

```

The cost profile is exactly zero on-chain — no MicroAlgos, no Wormhole verification, no opcode budget. The integrity profile is also exactly zero: a Hermes operator (or a compromised CDN, or a corrupt HTTP path) can return anything, since the client never verifies the guardian signatures on the VAA. **Use this tier for UI display, mindX agent reasoning, off-chain risk scoring, and BANKON debasement-dashboard graphs. Do not use it to gate funds.**

Tier 1 — Off-chain VAA verification with on-chain commit (cheap, partial integrity)

A useful middle tier that does not require any new Algotpy receiver is to (a) fetch the Hermes VAA off-chain, (b) verify Wormhole guardian signatures off-chain using @certusone/wormhole-sdk or the Python wormhole-sdk analogue, (c) parse the Pyth Merkle root and price message off-chain, and then (d) write the resulting (price, conf, expo, publish_time) plus the VAA hash into a DELTAVERSE-controlled Algorand application's box storage with a single ABI call signed by a privileged oracle-relayer key. This is roughly equivalent to a "trusted relayer" pattern

and is what many smaller Algorand integrations end up doing, because the cost of submitting and verifying a full Wormhole VAA on Algorand is non-trivial.

The integrity property here is "as good as your relayer key plus Hermes signature checks." The relayer key becomes a custodial trust assumption. This is acceptable for **non-adversarial pricing** (e.g., DebasementIndexV2 update cadence of ~15 minutes where the worst-case price corruption is bounded by the next refresh) but not for liquidation engines.

Tier 2 — On-chain Wormhole-VAA verification, DELTAVERSE custom Pyth receiver (canonical pull pattern, full integrity)

This is the architecturally-correct, fund-gating tier. Because Pyth never shipped an Algorand receiver, you build one. The components are:

1. The user (or a relayer paid by the user) fetches the price update VAA from Hermes, identical to step 1 in Tier 0.
2. The user assembles a transaction group containing:
 - Several `verifySigs` application calls to the Wormhole Core app (`842125965` on mainnet, `86525623` on testnet) that progressively check the 13/19 guardian signatures, paired with `nop` opup transactions to pool the opcode budget. The pattern is exactly the one Wormhole's own Algorand Token Bridge uses — see https://github.com/wormhole-foundation/wormhole/blob/main/algorand/wormhole_core.py for the `verifyVaa` flow and https://github.com/wormhole-foundation/wormhole/blob/main/algorand/token_bridge.py for an example consumer transaction group. (GitHub)
 - A final call to your **DELTAVERSE Pyth receiver app** whose Algopy code (a) re-checks the same VAA hash was verified earlier in the group by inspecting Wormhole Core's per-VAA `TmplSig` sequence-number storage, (b) parses the Pyth wire format from the verified VAA payload (Merkle root + Merkle proofs + price messages), (c) extracts `(feed_id, price, conf, expo, publish_time)` for each price message, and (d) writes them to box storage keyed by feed ID.
3. Downstream DELTAVERSE contracts read prices via an ABI `get_price_no_older_than(feed_id, max_age) -> (price, conf, expo)` method that returns the box-stored value if `Global.latest_timestamp - publish_time <= max_age` and otherwise reverts.

Because Pyth has not provided a reference implementation for Algorand, every line of the Pyth-payload-parsing code has to be written from scratch. The wire format you must reimplement is documented across the Pyth payload reference at <https://docs.pyth.network/price-feeds/pro/payload-reference> and is most easily ported from the Solidity reference at <https://github.com/pyth-network/pyth->

[crosschain/blob/main/target_chains/ethereum/contracts/contracts/pyth/PythUpgradable.sol](https://github.com/pyth-network/pyth-crosschain/blob/main/target_chains/ethereum/contracts/contracts/pyth/PythUpgradable.sol)) and the Move reference at (https://github.com/pyth-network/pyth-crosschain/tree/main/target_chains/aptos/contracts/sources). The TON receiver at (https://github.com/pyth-network/pyth-crosschain/tree/main/target_chains/ton/contracts) is also a good high-level template because, like Algorand, TON uses a non-EVM execution model and the Pyth team wrote it relatively recently.

Cost model on Algorand

The Wormhole VAA verification dominates cost. A 13-signature VAA requires roughly 4-6 transactions in the verification group on Algorand, each at the protocol minimum fee of 1000 μ Algos, plus inner-txn fee pooling for opup budget. Empirically (per the Wormhole Token Bridge case), a single VAA verification on Algorand mainnet costs in the range of 6,000–10,000 μ Algos in fees, plus the one-time MBR for the per-VAA sequence-number TmplSig account if it's a new 16k-bit block (~285,000 μ Algos minimum balance reservation that is paid once and then reused across thousands of subsequent VAAs in the same emitter/sequence window). Box storage rent on Algorand follows the formula $(2500 + 400 \times (\text{key_len} + \text{value_len}))$ μ Algos per box (see (<https://developer.algorand.org/docs/get-details/dapps/smart-contracts/apps/state/>)), so a price box keyed by 32-byte feed ID with a 32-byte value (price+conf+expo+publish_time packed) is about $(2500 + 400 \times 64 = 28,100)$ μ Algos in MBR per feed — paid once at creation.

For DELTAVERSE running, say, 12 fiat feeds plus BTC/USD plus ALGO/USD plus USDC/USD = 15 boxes, total one-time MBR is roughly 422,000 μ Algos. Per-update operational cost (per VAA, amortized over the feeds packed in a single Merkle root) is dominated by the ~10,000 μ Algos verification fee.

Staleness and confidence-interval checks

Pyth's Best Practices doc at (<https://docs.pyth.network/price-feeds/core/best-practices>) is unambiguous: every consumer must enforce a staleness bound via `get_price_no_older_than(feed_id, max_age)` and a confidence-bound check via `conf / price < threshold`. The exact thresholds are application-dependent — Pyth recommends `max_age ≤ 60 s` for liquidation logic and rejection if `conf/price > 0.005` (50 bps) for risk-bearing decisions. In Algorand this is straightforward: (Pyth)

```
python
```

```
# deltaverse/oracle/pyth_consumer.py - Algorand Python (Algopy)
from algopy import ARC4Contract, Box, Global, UInt64, Bytes, arc4

class PythConsumer(ARC4Contract):
    # 32-byte feed_id -> 32-byte packed (price:int64, conf:uint64, expo:int32, publi
    prices: Box[Bytes]

    @arc4.abimethod
    def get_price_no_older_than(self, feed_id: Bytes, max_age: UInt64) -> arc4.Tuple
        raw = self.prices[feed_id]
        price      = arc4.Int64.from_bytes(raw[0:8])
        conf       = arc4.UInt64.from_bytes(raw[8:16])
        expo       = arc4.Int32.from_bytes(raw[16:20])
        publish_t  = arc4.UInt64.from_bytes(raw[20:28]).native
        assert Global.latest_timestamp - publish_t <= max_age, "stale price"
        # Confidence guard: reject if conf/price > 50 bps (0.005)
        # Using integer math: conf * 200 > |price|
        p_abs = price.native if price.native >= 0 else -price.native
        assert conf.native * UInt64(200) <= UInt64(p_abs), "conf too wide"
        return arc4.Tuple((price, conf, expo))
```

The `update_price_feeds` companion method on the receiver is the heavy one — it accepts the raw VAA bytes, re-asserts that the previous transaction(s) in the group already invoked Wormhole Core's `verifyVaa` for this same VAA hash, parses the Pyth payload, and writes the box. The cleanest implementation pattern is to have your Algopy contract require that `gtxn[i]` for `i < this.txn.group_index` is an `appl` call to Wormhole Core App `842125965` with the matching VAA bytes as `application_args[1]` — replicating the pattern used inside `wormhole-foundation/wormhole/algorand/token_bridge.py`.

Tier 3 — Highest integrity: confidence-hardened pull + dual-oracle cross-check + TWAP smoothing

For DebasementIndexV2 and BitcoinAnchorOracle, Tier 2 alone is insufficient because (a) Pyth is one party — even with sub-second updates a single oracle failure can corrupt your index, (b) confidence intervals can widen during volatile markets exactly when DELTAVERSE most needs to trust the price, and (c) Algorand block latency means the Pyth update you commit on-chain is already several seconds old by the time your downstream contract reads it. Hardening means:

1. **Dual-oracle cross-check.** Read BTC/USD from Pyth (via your Tier-2 receiver) and from a second source — either a Folks Finance oracle (Folks publishes feeds for the assets it lists; see their integration repos under <https://github.com/Folks-Finance>) or a Tinyman v2 TWAP. The mainnet Tinyman router and pool contract IDs are documented at

<https://docs.tinyman.org/>), and the standard pattern is to compute a 30-minute time-weighted price from pool reserves and compare it to Pyth. Reject the trade if $\frac{|pyth - alt|}{pyth} > 1\%$.

2. **TWAP smoothing of Pyth itself.** Maintain a small ring buffer (8-16 entries) of recent Pyth prices in box storage and use the median rather than the latest spot value for the actual debasement-index update. This neutralizes single-block manipulation.
3. **Hard staleness reject.** For DebasementIndexV2 a 60-second `max_age` is appropriate; for BitcoinAnchorOracle anchoring real-world trades a tighter 15-second bound is safer.
4. **Confidence-ratio reject.** As in the code above, reject `conf/price > 50 bps`. During market dislocations (FX, equities holidays, low-liquidity altcoins) Pyth confidence intervals legitimately widen — see <https://docs.pyth.network/price-feeds/core/best-practices> — and your contract should treat that as "no signal" rather than as a signal.
(Pyth)
5. **No Pyth Lazer on Algorand.** Pyth Lazer / Pyth Pro is not deployable on Algorand (<https://github.com/pyth-network/pyth-crosschain/tree/main/lazer/contracts> shows EVM, Solana, Sui, Fogo only). If you genuinely need sub-100 ms latency, the right architecture is to run a Solana-side Lazer consumer and bridge the *decision* (not the price) to Algorand via Wormhole NTT or by signed off-chain attestation. Algorand block time — 2.82 seconds per the Algorand Developer Portal (<https://dev.algorand.co/concepts/transactions/blocks/>) following the Dynamic Round Times upgrade activated on mainnet January 17, 2024 — is itself the binding latency floor regardless of upstream oracle speed. (Algorand) (Binance)

Tier 4 — Ancillary: Pyth Entropy, OIS staking, PYTH bridging

Pyth Entropy is unavailable on Algorand. The repository at https://github.com/pyth-network/pyth-crosschain/tree/main/target_chains/ethereum/contracts/contracts/entropy is EVM-only and the contract address list at <https://docs.pyth.network/entropy/contract-addresses> enumerates only EVM chains. For DAIO randomness on Algorand, the architecturally correct substitute is the AVM's own `vrf_verify` opcode plus the Algorand Foundation's Randomness Beacon application — per the Algorand Developer Portal "Usage and Best Practices for Randomness Beacon" (<https://developer.algorand.org/articles/randomness-on-algorand/>), the MainNet randomness beacon smart contract App ID is `1615566206`, and the TestNet App ID is `110096026`. (Third-party usage examples occasionally cite `947957720` as the mainnet ID; verify on-chain before integrating.) Cross-chain Pyth Entropy from EVM into Algorand is possible via a Wormhole message but the latency and cost make it inferior to native VRF.

Oracle Integrity Staking (OIS) backs Pyth's publisher network with slashable stake and is exclusively a Solana/Pythnet program: the staking UI is <https://staking.pyth.network/> and

the program source is in (<https://github.com/pyth-network/governance>) (also see <https://github.com/pyth-network/staking>). To participate, DELTAVERSE must hold native PYTH on Solana. Pyth's token-address registry at (<https://docs.pyth.network/home/pyth-token/pyth-token-addresses>) shows no Algorand ASA for PYTH; the Wormhole Token Bridge on Algorand (App [842126029](#)) could in principle attest a wrapped PYTH ASA, but none is canonical. The recommended PYTH acquisition path for an Algorand-native protocol is: acquire ALGO on Algorand → bridge to Solana via Portal (<https://portalbridge.com>) or Allbridge → swap to PYTH on Jupiter/Raydium → stake via the OIS staking UI. Treat the resulting PYTH staking exposure as a Solana-side position with its own custody requirements; do not try to mirror it onto Algorand.

Tier 5 — Algorand-native alternatives for cross-check or fallback

For comparison and for the dual-oracle cross-check in Tier 3:

- **Folks Finance Oracle** — the lending protocol's price oracle. Their public Algorand/Wormhole integration code lives at (<https://github.com/Folks-Finance/algorand-wormhole-example>) and their core SDK at (<https://github.com/Folks-Finance/folks-finance-js-sdk>). Folks publishes prices for its supported assets via on-chain feeds with their own update cadence; the integration pattern is to read their app's box/global state directly from a downstream contract.
- **Goracle** — Algorand's general-purpose oracle network at (<https://gora.io>). SDK and contract source at (<https://github.com/GoracleNetwork>). Goracle is request-response style (more like Chainlink Functions than Pyth's pull) and is suited to non-price external data (weather, sports, REST APIs).
- **Tinyman v2 TWAP** — derived from pool reserves at (<https://github.com/tinymanorg/tinyman-py-sdk>) (Python SDK) and the v2 contracts at (<https://github.com/tinymanorg/tinyman-contracts-v2>). Compute TWAP off the cumulative-price oracle exposed by Tinyman v2 pools.
- **Pact and Vestige** — Pact's Python SDK at (<https://github.com/pactfi/pact-py-sdk>) exposes pool state directly; Vestige (<https://vestige.fi>) aggregates DEX prices and can be used as an off-chain reference for sanity-checks.

None of these is a like-for-like Pyth replacement (Pyth's first-party publisher network is genuinely differentiated), but for cross-check purposes they suffice.

A concrete Algopy sketch for DELTAVERSE BitcoinAnchorOracle / DebasementIndexV2

The full integration sketch, assuming Python 3.12 + Algopy + AlgoKit:

```
python
```

```

# deltaverse/oracle/pyth_receiver.py
"""
DELTAVERSE Pyth Receiver – verifies Pyth-Wormhole VAAs through Wormhole Core
(mainnet App ID 842125965) and persists (price, conf, expo, publish_time) per
feed ID in box storage. Downstream contracts (DebasementIndexV2,
BitcoinAnchorOracle) consume via get_price_no_older_than().
"""
from algopy import (
    ARC4Contract, Box, BoxMap, Bytes, Global, UInt64, Txn, gtxn,
    Application, arc4, op,
)

WORMHOLE_CORE_APP_ID = UInt64(842125965) # mainnet; use 86525623 for testnet
# Pythnet emitter (Wormhole chain 26) – the only emitter we accept VAAs from.
PYTHNET_EMITTER_CHAIN = UInt64(26)
PYTHNET_EMITTER_ADDR = Bytes.from_hex( # Pyth's Wormhole emitter on Pythnet
    "f8cd23c2ab91237730770bbea08d61005cdda0984348f3f6eecb559638c0bba0"
)

class PythReceiver(ARC4Contract):
    # Per-feed price record: 8B price (i64) | 8B conf (u64) | 4B expo (i32) | 8B pub
    prices = BoxMap(Bytes, Bytes, key_prefix=b"p_")

    @arc4.abimethod
    def update_price_feeds(self, vaa: arc4.DynamicBytes) -> None:
        """
        Caller MUST place a Wormhole Core `verifyVaa` group earlier in this
        atomic transaction group, passing the same `vaa` bytes. We verify the
        group structure here.
        """
        # 1. Walk preceding group txns and assert at least one was a verifyVaa
        #    call to Wormhole Core with this VAA's bytes.
        found = UInt64(0)
        i = UInt64(0)
        while i < Txn.group_index:
            t = gtxn.ApplicationCallTransaction(i)
            if (t.application_id.id == WORMHOLE_CORE_APP_ID
                and t.app_args(0) == Bytes(b"verifyVAA")
                and t.app_args(1) == vaa.bytes):
                found = UInt64(1)
            i = i + UInt64(1)
        assert found == UInt64(1), "VAA not verified in group"

```

```

# 2. Parse VAA header to extract (emitter_chain, emitter_address, payload).
body = self._parse_vaa(vaa.bytes) # returns (emitter_chain, emitter_addr, p
assert body.emitter_chain == PYTHNET_EMITTER_CHAIN, "wrong chain"
assert body.emitter_addr == PYTHNET_EMITTER_ADDR, "wrong emitter"

# 3. Parse Pyth Merkle-rooted accumulator payload from body.payload.
#   For each price message inside: write (price, conf, expo, pub_time)
#   keyed by feed_id into the `prices` box.
self._apply_pyth_payload(body.payload)

```

```

@arc4.abimethod(readonly=True)
def get_price_no_older_than(
    self, feed_id: arc4.StaticBytes[32], max_age: UInt64
) -> arc4.Tuple[arc4.Int64, arc4.UInt64, arc4.Int32]:
    raw = self.prices[feed_id.bytes]
    price = arc4.Int64.from_bytes(op.substring(raw, 0, 8))
    conf = arc4.UInt64.from_bytes(op.substring(raw, 8, 16))
    expo = arc4.Int32.from_bytes(op.substring(raw, 16, 20))
    pub_t = arc4.UInt64.from_bytes(op.substring(raw, 20, 28)).native
    assert Global.latest_timestamp - pub_t <= max_age, "stale"
    return arc4.Tuple((price, conf, expo))

```

```

# _parse_vaa and _apply_pyth_payload elided – see the EVM reference at
# https://github.com/pyth-network/pyth-crosschain/blob/main/target_chains/
#   ethereum/contracts/contracts/pyth/PythUpgradable.sol
# and the TON port at ../target_chains/ton/contracts for byte-for-byte
# reference of the wire format.

```

```

# deltaverse/contracts/debasement_index_v2.py

```

```

class DebasementIndexV2(ARC4Contract):
    """
    Pulls BTC/USD plus a basket of fiat feeds and emits a debasement index.
    Hard staleness and confidence guards. Dual-oracle cross-check vs Folks.
    """
    pyth_app: UInt64 # Application ID of PythReceiver
    folks_app: UInt64 # Application ID of Folks oracle
    last_index: UInt64
    last_update: UInt64

    BTC_USD = Bytes.from_hex(
        "e62df6c8b4a85fe1a67db44dc12de5db330f7ac66b72dc658afedf0f4a415b43"
    )
    # ALGO_USD, EUR_USD, JPY_USD, GBP_USD, XAU_USD, USDC_USD ... resolved at deploy

```

```

@arc4.abimethod
def update_index(self) -> arc4.UInt64:
    # 1. Read BTC/USD from Pyth with 60s max age, 50bps conf bound.
    btc_price, btc_conf, btc_expo = self._pyth_price(self.BTC_USD, UInt64(60))
    # 2. Cross-check against Folks BTC oracle; reject if delta > 1%.
    btc_folks = self._folks_price(self.BTC_USD)
    delta_bps = self._abs_delta_bps(btc_price.native, btc_folks)
    assert delta_bps <= UInt64(100), "dual-oracle mismatch"

    # 3. Pull fiat basket, compute debasement index = BTC / basket_weighted_fiat
    # ... (basket math omitted for brevity)

    self.last_index = ...
    self.last_update = Global.latest_timestamp
    return arc4.UInt64(self.last_index)

def _pyth_price(self, feed_id: Bytes, max_age: UInt64):
    return arc4.abi_call(
        "get_price_no_older_than(byte[32],uint64)(int64,uint64,int32)",
        feed_id, max_age,
        app_id=self.pyth_app,
    )

```

The off-chain Python driver that assembles the update transaction group (using `algokit-utils-py` from <https://github.com/algorandfoundation/algokit-utils-py>) fetches the latest VAA from Hermes, splits it into the signature batches that Wormhole Core expects, builds the `verifyVaa` group of 4-6 `appl` calls plus `opup` `nop` transactions for opcode budget, appends the `update_price_feeds` call to the DELTAVERSE PythReceiver, and finally appends the `update_index` call to `DebasementIndexV2`. The full group is submitted as a single atomic transaction. A typical successful update lands in one Algorand block (~2.82 s) at a total fee in the range of 10,000-15,000 μ Algos (~0.012 ALGO).

Recommendations

- Tier 0 today, immediately.** Ship `pythai/oracle/hermes.py` as shown above and wire it into mindX agent reasoning and the BANKON dashboards within 24 hours of receiving this report. Zero on-chain cost, full Pyth feed catalog, sub-second latency. This unblocks the AI-side product while the on-chain receiver is being built.
- Tier 1 in week 1 as a stopgap for non-adversarial settlement.** Stand up an off-chain Hermes-fetcher + Wormhole-SDK verifier (off-chain) + Algorand relayer key that writes signed `(price, conf, expo, publish_time)` rows into a DELTAVERSE-owned

application. This gets you fund-gating prices on-chain in a few days at the cost of one custodial key, which you mitigate with a 2-of-3 multisig relayer. Acceptable for DebasementIndexV2 v0 with 15-minute refresh.

3. **Tier 2 over weeks 2-4.** Port the Pyth payload parser from the TON or Aptos reference into Algopy, integrate with Wormhole Core App (842125965), audit. Take the time to copy the per-VAA-sequence TmplSig MBR pattern from (wormhole-foundation/wormhole/algorand/wormhole_core.py) exactly — it is non-trivial and any bug there is a replay-attack vector. Budget ~6 weeks of senior engineering plus an audit; the audit firm should be one of the Pyth-recommended auditors (see (https://github.com/pyth-network/audit-reports)).
4. **Tier 3 hardening before any liquidation engine goes live.** Layer the TWAP buffer, dual-oracle cross-check with Folks or Tinyman, and 50 bps confidence-ratio reject on top of the Tier-2 receiver. Reject prices rather than pass through with widening confidence.
5. **Do not wait for Pyth to deliver Algorand.** Pyth's contract-addresses registry has been static for Algorand since the 2022 announcement and there is no open PR or roadmap commitment to Algorand in (pyth-network/pyth-crosschain). Treat Algorand-side Pyth as a build-it-yourself problem.

Benchmarks that would change these recommendations: if Pyth ships an official (target_chains/algorand/) directory in (pyth-network/pyth-crosschain) with a deployed App ID, immediately deprecate the custom DELTAVERSE PythReceiver and migrate to the official one. If Algorand Foundation deploys a partnership-blessed Wormhole-Pyth Algorand receiver (analogous to what Aptos has at (0x7e783b349d3e89cf5931af376ebadbfbab855b3fa239b7ada8f5a92fbea6b387)), migrate. If Pyth Lazer adds Algorand to the lazer/contracts tree, evaluate it for BitcoinAnchorOracle sub-second pricing. (Aptos)

Caveats

- **The "Pyth on Algorand" surface is unofficial.** No on-chain Pyth Algorand contract is published by the Pyth Network team. Anything you build is your own and inherits its full security boundary — Pyth offers no audit, no SLA, no Hermes-side filtering specifically for Algorand consumers.
- **C3.io's integration is closed-source.** The architecture described here is inferred from C3 and Pyth blog posts; we did not have direct access to C3's Health Calculator contract source.
- **Wormhole guardian trust assumption.** The Tier-2/3 design inherits Wormhole's 13-of-19 guardian assumption. The Wormhole exploit on February 2, 2022 is a reminder this is a real attack surface — per Chainalysis ((https://www.chainalysis.com/blog/wormhole-hack-february-2022/)) the attacker exploited a deprecated, insecure signature-verification function to mint and steal 120,000 wETH worth over \$320 million, the second-largest DeFi

hack at the time. If you need full trust minimization, no oracle in this map provides it on Algorand today. (Halborn) (Merkle Science)

- **Feed-ID drift.** The price-feed IDs listed in this report are Stable-channel mainnet values as of May 2026 and should be re-verified at integration time via https://hermes.pyth.network/v2/price_feeds. Pyth occasionally rotates feed IDs at major version boundaries, and Beta-channel IDs differ from Stable-channel IDs for the same logical pair. (Pyth)
- **Algorand cost numbers are estimates.** The 10,000–15,000 μ Algos per VAA update figure is consistent with Wormhole Token Bridge group sizes on Algorand but should be benchmarked on testnet (Wormhole Core App (86525623)) before any mainnet commitment, because the exact opup pooling and box-MBR mix is workload-dependent.
- **Randomness Beacon App ID discrepancy.** The Algorand Developer Portal states the MainNet randomness beacon App ID is (1615566206), while some third-party integrations (e.g., AlgoDice on GitHub) cite (947957720). Verify on-chain via Pera Explorer before wiring DAIO randomness to a specific App ID. (Algorand)
- **Pyth Pro / Lazer for Algorand may never ship.** Do not architect around the assumption that Algorand will get sub-100ms Lazer feeds; design BitcoinAnchorOracle around Algorand's native block latency floor.